Day 2 Master Handbook: Machine Learning & Neural Networks

Comprehensive Theoretical Reference & Technical Study Guide

1. AI vs. Machine Learning vs. Deep Learning

• **Artificial Intelligence (AI):** Any system mimicking human decision-making. Does NOT require learning (e.g. 1990s chess engines with hard-coded if/else rules).
• **Machine Learning (ML):** Algorithms discovering predictive rules from past data: *Data* + *Past Answers* → *Machine Learns Rules*. Requires human feature engineering.
• **Deep Learning (DL):** Multi-layered neural networks performing **Automatic Feature Extraction** on raw unstructured sensory inputs (images, audio, text).

2. Learning Paradigms & Task Taxonomy

• **Supervised Learning:** Ground truth target labels (y) exist. Learns mapping f(X) → y. Tasks: **Regression** (continuous numbers like Price, Salary) and **Classification** (discrete categories like Pass/Fail, Cat/Dog).
• **Unsupervised Learning:** NO target labels exist. Discovers latent geometry, groupings, and distributions: P(X). Tasks: **Clustering** (K-Means) and **Dimensionality Reduction** (PCA).

3. The 5 Core Algorithms Master Comparison

Algorithm	Type	Mathematical Mechanism	Strengths	Weaknesses
Linear Regression	Supervised (Regression)	Ordinary Least Squares (OLS) minimizing RSS: y = wX + b	Highly interpretable, fast	Assumes strictly linear relations
Logistic Regression	Supervised (Classification)	Bends line via Sigmoid into probability: P = 1/(1+e^-z)	Outputs calibrated probability	Linear decision boundary only
Decision Tree	Supervised (Both)	Partitions feature space via Gini Impurity or Entropy	Zero scaling needed, interpretable	Severe Overfitting if unpruned
Random Forest	Supervised (Both)	Bagging ensemble of 100+ de-correlated Trees + feature sub-sampling	State-of-the-art on tabular data	Slow on massive real-time data
K-Means	Unsupervised (Clustering)	Iteratively minimizes WCSS distance to K geometric centroids	Scalable, discovers groups	Assumes spherical clusters

4. Model Diagnostics & Evaluation Metrics

The Bias-Variance Tradeoff, K-Fold Validation, and Mathematical Scoring

The Bias-Variance Tradeoff & Overfitting

- **High Bias (Underfitting):** Model is too simple. Fails on both training and test data. Fix: Increase model capacity, add features.
- **High Variance (Overfitting):** Model memorizes training noise (e.g. 99% train score, 55% test score). Fix: Regularization, more data, Random Forests, feature reduction.
- **K-Fold Cross-Validation:** Splits data into K folds (e.g. 5 folds). Rotates test fold 5 times and averages scores to prevent split luck.

Regression Metrics (Measuring Numerical Error)

Metric	Formula / Concept	Interpretation	Outlier Sensitivity
MAE	Mean(|y_true - y_pred|)	Average mistake magnitude in actual dollars/units	Robust to outliers
MSE	Mean((y_true - y_pred)^2)	Squares errors before averaging	Highly sensitive
RMSE	Sqrt(MSE)	Converts squared penalty back into original units	Penalizes large errors
R^2 Score	1 - (RSS / TSS)	Proportion of target variance explained (0.0 to 1.0)	Relative benchmark

Classification Metrics (Confusion Matrix & Tradeoffs)

Metric	Formula	Real-World Meaning	When to Prioritize
Accuracy	(TP + TN) / Total	Percentage of overall correct guesses	Balanced classes only
Precision	TP / (TP + FP)	Of all positive alarms called, how many were real?	When False Alarms are costly (Spam)
Recall	TP / (TP + FN)	Of all actual positive cases, how many did we catch?	When Missed Cases are fatal (Cancer, Fraud)
F1-Score	2*(Prec*Rec)/(Prec+Rec)	Harmonic mean balancing Precision and Recall	Imbalanced datasets
ROC-AUC	Area Under TPR vs FPR	Model ability to separate classes across all thresholds	Overall diagnostic evaluation

5. Neural Networks, Activation Functions & Optimization

The Architecture of Modern Deep Learning: Neurons, ReLU, Loss, and Gradient Descent

The Artificial Neuron (Perceptron)

An artificial neuron mimics biological synapses via a linear weighted combination followed by an activation:

1. **Weighted Sum:** z = (w1*x1) + (w2*x2) + ... + (wd*xd) + b
2. **Activation:** output = f(z)

Why Non-Linear Activations are Mandatory: Stacking 50 purely linear layers collapses mathematically into a single linear matrix multiplication: W2 * (W1 * x) = W_combined * x. Without non-linear activation functions, a deep neural network is just Linear Regression and can never learn curved boundaries, circles, or image patterns!

Activation Functions Deep Dive

Function	Formula	Range	Mechanisms & When to Use
ReLU	f(z) = max(0, z)	[0, +inf)	The king of hidden layers. Turns negatives to 0, passes positives. Solves vanishing gradient, ultra-fast.
Sigmoid	f(z) = 1 / (1 + e^-z)	[0, 1]	Smooth S-curve. Standard output layer for Binary Classification (outputs probability). Causes vanishing gradient in
Tanh	f(z) = (e^z - e^-z)/(e^z + e^-z)	[-1, 1]	Zero-centered S-curve. Preferred over Sigmoid in recurrent networks.
Softmax	e^zi / Sum(e^zj)	[0, 1] (Sum=1)	Multi-class classification output layer. Normalizes raw logits into mutually exclusive percentage distribution.

Loss Functions & Gradient Descent Optimization

- **Loss Function:** Mathematical penalty score. Regression uses **MSE**; Binary classification uses **Binary Cross-Entropy (Log Loss)**: Loss = -[y*log(p) + (1-y)*log(1-p)].
- **Gradient Descent:** Iterative optimization algorithm finding the lowest valley of loss. Updates weights in the opposite direction of the gradient:
  New_Weight = Old_Weight - (Learning_Rate * Gradient)
- **Learning Rate (Step Size):**
  - *Too Large:* Steps overshoot the valley bottom; loss diverges erratically (Loss → NaN).
  - *Too Small:* Takes glacial baby steps; network takes days to converge or gets trapped.
- **Backpropagation:** Applies the Chain Rule of calculus from output backwards through every layer to compute exact weight gradients.